

DevOps Beginner Projects — Resume Friendly

Quick summary table

#	Project	Core Skills	Approx. Time	Resume-ready deliverable
1	CI/CD pipeline for a sample web app	Git, Jenkins/GitHub Actions, Docker	1–2 weeks	Built CI/CD pipeline automating build, test, and deploy for Node/Flask app
2	Infrastructure as Code with Terraform (multi-tier app on cloud)	Terraform, AWS/GCP, VPC, EC2/RDS	2–3 weeks	Provisioned reproducible infrastructure using Terraform for a multi-tier web app
3	Container orchestration with Kubernetes (k8s)	Docker, Kubernetes, Helm, kubectl	2–3 weeks	Deployed microservices to Kubernetes with Helm charts and autoscaling
4	Monitoring + Alerting stack for an app	Prometheus, Grafana, Alertmanager, exporters	1–2 weeks	Implemented monitoring and alerting for services using Prometheus and Grafana
5	GitOps workflow using Flux/ArgoCD	GitOps, Flux or ArgoCD, K8s, CI	1–2 weeks	Implemented GitOps deployment pipeline to manage cluster state from Git

Project 1 — CI/CD pipeline for a sample web app

Goal: Automate building, testing and deploying a simple web app (Node.js or Flask) whenever code is pushed.

Why it's resume friendly: Demonstrates automation, testing, containerization, and a production-style workflow.

Tools & Tech: GitHub (or GitLab), GitHub Actions (or Jenkins), Docker, Docker Hub (or GitHub Container Registry), a simple cloud VM or Heroku for deployment, unit test framework (jest/pytest).

Step-by-step build process:

1. Create the sample app

- Scaffold a minimal web app (e.g., Node + Express or Python + Flask) with one API endpoint and a README.
- Add unit tests (one or two) using jest or pytest.

2. Initialize Git & GitHub repo

- `git init`, add `.gitignore`, commit code, push to GitHub.

3. Write Dockerfile

- Create a small, production-friendly Dockerfile (use a lightweight base image, copy app, install deps, set `CMD`).
- Build and run locally to verify: `docker build -t myapp:local .` and `docker run -p 5000:5000 myapp:local`.

4. Set up CI (GitHub Actions)

- Add `.github/workflows/ci.yml` that runs on `push` and `pull_request`:
 - Steps: checkout, set up node/python, install deps, run tests, build Docker image, optionally run `docker build` as a smoke test.

5. Push Docker image to registry

- In CI, login to Docker Hub / GHCR and push the built image with a tag like `${{github.sha}}`.

6. Add CD pipeline

- Option A (simple): Use GitHub Actions to deploy to Heroku or a VM via SSH after successful tests and image push.
- Option B (container): Deploy the pushed Docker image to a cloud VM (docker-compose) or to a simple Kubernetes cluster.

7. Add badges and README

- Add build/test badges to README and explain the pipeline steps.

Deliverables to push to GitHub: source code, unit tests, Dockerfile, `.github/workflows/ci.yml`, deployment script, README with architecture diagram.

Resume bullets (examples):

- Built end-to-end CI/CD pipeline using GitHub Actions, Docker, and Docker Hub to automate build, test and deployment of a Node/Flask app.
- Reduced manual deployment steps and ensured green-test-only releases using automated test gating.

Stretch goals: add canary deploys, blue/green, run integration tests in pipeline, or add security scanning (Snyk/Trivy).

Project 2 — Infrastructure as Code with Terraform (multi-tier app)

Goal: Provision a reproducible multi-tier environment (VPC, subnets, app servers, database, security groups) in AWS or GCP using Terraform.

Why it's resume friendly: Shows cloud infra skills, IaC discipline, and environment reproducibility.

Tools & Tech: Terraform, AWS (or GCP/Azure), Terraform Cloud or remote backend (S3 + DynamoDB for AWS), optional Ansible for configuration.

Step-by-step build process:

1. Design the architecture

- Simple 3-tier: public load balancer, private app servers (ASG/EC2), private database (RDS). Diagram it.

2. Set up a repo and Terraform structure

- Create `modules/` for `network`, `compute`, `database`, and root `envs/dev` folder for environment variables.
- Use `main.tf`, `variables.tf`, `outputs.tf` per module.

3. Configure remote state

- Configure S3 backend + DynamoDB locking (AWS) or Terraform Cloud workspace.

4. Implement network module

- Create VPC, public/private subnets, route tables, NAT gateway.

5. Implement compute module

- Launch template or EC2 autoscaling group in private subnets, create an IAM role, and user-data to pull container or code.

6. Implement database module

- Create RDS instance with subnet group and parameter group; ensure security group is locked down to app SG.

7. Outputs and variables

- Expose endpoints, DB host, security group IDs as outputs.

8. Apply & test

- `terraform init`, `terraform plan`, `terraform apply` in a dev workspace. Validate connectivity by deploying the sample app from project 1.

9. Destroy & repeatability

- Test `terraform destroy` and re-apply to ensure idempotency.

Deliverables: Terraform repo, modules, README with state backend instructions and diagrams.

Resume bullets:

- Created modular Terraform code to provision a secure multi-tier environment (VPC, ASG, RDS) with remote state and locking.
- Implemented environment promotion by separating `dev` and `prod` workspaces and parameterized variables.

Stretch goals: add CI to run `terraform fmt/validate/plan` checks; integrate with policy checks (Sentinel, OPA/Gatekeeper).

Project 3 — Container orchestration with Kubernetes (deploy microservices)

Goal: Containerize a small microservice app (2 services + DB), deploy to Kubernetes cluster with Helm charts, and implement autoscaling and rolling updates.

Why it's resume friendly: Demonstrates modern deployment practices, Helm, scaling and service discovery.

Tools & Tech: Docker, Kubernetes (Minikube/kind for local or EKS/GKE for cloud), Helm, kubectl, Horizontal Pod Autoscaler, Persistent Volumes.

Step-by-step build process:

1. Prepare microservices

- Two simple services (e.g., `frontend` + `backend`) that communicate over HTTP, and use a shared Redis or Postgres.

2. Containerize services

- Create Dockerfiles for each service and push images to a registry.

3. Create k8s manifests or Helm chart

- Write `Deployment`, `Service`, `ConfigMap`, `Secret`, `Ingress` manifests or a Helm chart that templatiszes values.

4. Run a local cluster

- Use `kind` or `minikube` for local testing. `kubectl apply -f` or `helm install` the chart.

5. Configure autoscaling and liveness/readiness

- Add `readinessProbe`/`livenessProbe` and create HPA based on CPU or custom metrics.

6. Set up Ingress and TLS

- Use an ingress controller (nginx) and cert-manager (optional) for HTTPS.

7. CI/CD integration

- Add a GitHub Action to build images and update Helm release via `helm upgrade --install` or use a GitOps tool (project 5).

8. Observe and debug

- Use `kubectl get pods`, `kubectl logs`, and port-forward for debugging.

Deliverables: Helm chart, k8s manifests, Docker images, README with commands and architecture.

Resume bullets:

- Deployed microservices to Kubernetes using Helm; implemented autoscaling and health checks to improve resilience.

- Automated image builds and releases to the cluster using GitHub Actions.

Stretch goals: use a managed cluster (EKS/GKE), implement network policies, or enable Prometheus scraping.

Project 4 — Monitoring + Alerting stack for an application

Goal: Add observability to an app: metrics (Prometheus), dashboards (Grafana), logs (Loki/ELK), and alerts (Alertmanager) with a runbook.

Why it's resume friendly: Observability is critical in production — this shows you can instrument and operate systems.

Tools & Tech: Prometheus, Grafana, Alertmanager, node exporter/cadvisor, Grafana dashboards, optionally Loki/Fluentd for logs.

Step-by-step build process:

1. Instrument the app

- Expose metrics endpoint (Prometheus client library) and add basic metrics: request count, latency histograms.

2. Deploy Prometheus

- Use the Prometheus community Helm chart or manifests; configure `scrape_configs` for app and node exporters.

3. Deploy Grafana

- Add dashboards to visualize latency, error rates, instance metrics. Import community dashboards as starting point.

4. Set up Alertmanager

- Configure alert rules in Prometheus and routing/receivers in Alertmanager (email/Slack webhook).

5. Add logging

- Optional: push logs to Loki + Grafana or set up Fluentd/Fluent Bit to send logs to Elasticsearch.

6. Create runbooks

- For each alert, document steps to investigate and mitigate (check logs, scale pods, restart service).

7. Demo & screenshots

- Capture dashboard screenshots and examples of alerts triggered and resolved.

Deliverables: Prometheus config, Grafana dashboards (JSON), alert rules, runbook.md, README.

Resume bullets:

- Implemented end-to-end monitoring with Prometheus, Grafana and Alertmanager; created dashboards and alerting runbooks to reduce MTTR.
- Instrumented application code with metrics to enable SLO monitoring and capacity planning.

Stretch goals: implement SLOs and error budget tracking, add tracing with Jaeger.

Project 5 — GitOps workflow using Flux or ArgoCD

Goal: Manage Kubernetes deployments via Git: cluster state is driven by Git repositories and automatically reconciled.

Why it's resume friendly: Shows modern infrastructure management and team-friendly deployment workflows.

Tools & Tech: ArgoCD or Flux, GitHub, Kubernetes cluster, Helm charts or Kustomize.

Step-by-step build process:

1. Set up your cluster

- Use `kind`/minikube or a managed cluster. Ensure `kubectl` access.

2. Create two Git repos

- `app-code` (source code) and `infra-gitops` (deployment manifests/Helm values).

3. Install ArgoCD or Flux

- Follow quickstart to install into the cluster.

4. Push manifests to infra repo

- Add Helm values or customize overlays for `dev` and `prod` directories.

5. Connect ArgoCD/Flux to the repo

- Configure an Application (ArgoCD) or GitRepository + Kustomization (Flux) to watch the repo and sync.

6. Automate image updates

- Use Image Automation (Flux Image Update) or ArgoCD Image Updater to automatically create PRs or update manifests when new images are available.

7. Promote using PRs

- Use branches/PRs to promote from `dev` to `prod` and demonstrate automated sync in the cluster.

8. Observe and rollback

- Trigger a bad change and show how Git revert + automatic sync rolls back the cluster.

Deliverables: GitOps repo, instructions, demo GIFs/screenshots, example PRs showing promotions.

Resume bullets:

- Implemented GitOps using ArgoCD/Flux to declaratively manage Kubernetes cluster state and enable safe, auditable deployments.
- Automated image updates and promotion flows to reduce manual deployments and human errors.

Stretch goals: add policy checks (OPA/Gatekeeper), multi-cluster GitOps, or GitHub Actions to automate image tagging.

How to present this sheet to your followers

1. Convert the markdown to a shareable PDF or Google Doc. (Tip: GitHub renders the README nicely — you can also use [pandoc](#) or your editor to export to PDF.)
 2. For Instagram carousel: split each project into 1–2 slides: title, one-line summary, 3 key steps, and a final slide with resume bullets.
 3. For Twitter/X threads: post one tweet per project with a link to the repo and a sample resume bullet.
 4. For LinkedIn: write a short post with the 5-line summary table and attach the PDF.
-

Boilerplate: GitHub repo structure suggestion (for each project)

```
project-name/
├── app/           # source code
├── Dockerfile
├── helm/          # optional helm chart
├── infra/         # terraform or k8s manifests
├── .github/workflows/ # CI/CD pipeline
├── monitoring/    # prometheus/grafana configs
└── README.md
```

└─ runbook.md
